Supplementary Material:

Closing the Security Activation Gap: A Security-by-Design Methodology for IoT Ecosystems and Its Empirical Validation in a DC Smart Microgrid

S0. Generic Methodology Specification

The methodology's full definitional treatment (formal identity model, Root of Trust process, STRIDE-to-stage mapping) is given in the companion conceptual paper (**?**). Tables S1–S4 below give a condensed, technology-agnostic statement of what each numbered process requires or produces in general, independent of any specific ecosystem — so that this submission is self-contained for judging whether the SMG case study (Tables S5–S8) actually followed the methodology's own guidance, without requiring the companion paper in hand. Process IDs match those cited throughout the main text and Tables S5–S8.

S1. Methodology Parameterized Specification: SMG Case Study

This document provides the full parameterized specification of all four phases for the SMG case study, complementing the condensed traceability in the main text's Tables 10 and 11, and instantiating the generic specification in Tables S1–S4 above for this specific ecosystem. Each row in Tables S5–S8 corresponds to a methodology parameter defined in the methodology's process tables. Codes with an additional digit (e.g., 4.1.2.1) refine a stage-level process (as cited in the main text) into its implementation-specific components; the parent code is the methodology-level process that those refinements collectively satisfy.

References

Fagan, M., Marron, J., Brady, K., Cuthill, B., Megas, K., Herold, R., 2021. IoT Device Cybersecurity Guidance for the Federal Government: Establishing IoT Device Cybersecurity Requirements. NIST Special Publication 800-213. National Institute of Standards and Technology (NIST). Gaithersburg, MD. doi:10.6028/NIST.SP.800-213.

Stanicek, O., 2022. esp32_puflib: An SRAM-based physical unclonable function library for the ESP32 microcontroller family. Software library, ESP-IDF component, https://github.com/staniond/esp32_puflib.

UK Government Security, 2024. Secure by design principles. https://www.security.gov.uk/guidance/secure-by-design/principles/. Accessed: 30 April 2024.

Table S1: Phase A, Stage 1: Organizational aspects — generic specification.

Process ID	Parameter	Generic Requirement / Expected Output
1.1	Problem statement	Define the ecosystem's purpose, scope, and the operational problem it addresses, independent of any technology choice
1.1.1	Cost-benefit justification	Quantify expected costs (development, deployment, maintenance) against expected benefits (risk reduction, efficiency, compliance)
1.1.2	Entities involved	Enumerate every entity type that will participate (devices, servers, users, administrators, and any physical element tracked only indirectly)
1.3.1	Development iteration (TRL)	Select a target Technology Readiness Level for the current iteration and the criteria required to advance to the next
1.4.1	Criticality classification	Classify the ecosystem's criticality tier, determining the applicable security rigor
1.4.2	Technological sovereignty	Specify which parties control the technology stack (self-hosted vs. third-party) and the resulting trust boundary
1.4.3	Data sovereignty	Specify where, and under whose control, ecosystem data is stored and processed
1.4.4	Access control policy	Define the access-control model governing which entity types may perform which operations
1.4.5	Communication policy	Define mandatory communication-security properties, independent of the specific protocol selected at Stage 4
1.4.6	Applicable guidelines	Identify the external security/regulatory guidelines and standards the ecosystem must comply with

Table S2: Phase B, Stages 2–3: Architecture and service design — generic specification.

Process ID	Parameter	Generic Requirement / Expected Output
2.1.4	Communication topology	Select the topology (centralized, decentralized, mesh) consistent with the Stage 1.4.2/1.4.3 sovereignty requirements
2.1.5	Communication infrastructure	Specify the physical/link-layer infrastructure supporting the chosen topology
2.1.7	Processing topology	Specify where computation occurs (edge/fog/cloud) for each ecosystem function
2.1.8	Entity hierarchy	Define the privilege ordering among entity types
2.2.2	Authentication mechanism	For each entity type, specify the class of authentication mechanism (symmetric/asymmetric; what each party must prove) satisfying Eq. (1)–(3)'s authentication-parameter subset
2.2.3	Root of Trust process	Instantiate the methodology's generic Root of Trust process (enrollment, registration, per-session challenge, mutual identification) for the chosen entity types
2.2.4	Secure communications	Specify the data-channel confidentiality/integrity mechanism protecting entity-to-entity communication once the Root of Trust establishes a session
3.1.1	Services defined	Enumerate the services the ecosystem must provide and which entity types each one involves
3.2.1	Access control	Instantiate the Stage 1.4.4 policy as an enforceable mechanism for the chosen architecture
3.2.2	Identity storage	Specify, per entity type, where and how identity parameters (Eq. 1–3) are persisted, consistent with the Stage 3 secure-storage evaluation

Table S3: Phase C, Stages 4–5: Implementation technologies and functional elements — generic specification.

Process ID	Parameter	Generic Requirement / Expected Output
4.1.1	TRL criteria met	Define concrete, falsifiable criteria demonstrating the Stage 1.3.1 target TRL has been reached
4.1.2.1	Constrained-entity platform	Select the hardware/software platform for resource-constrained entities, evaluated against the Stage 3 secure-storage requirement
4.1.2.2	Cryptography	Select the concrete cryptographic primitives instantiating the Stage 2.2.2/2.2.3/2.2.4 mechanism classes
4.1.3	Communication protocol	Select the concrete application-layer protocol instantiating the Stage 2.1.4/2.1.5 topology and infrastructure choices
4.2.1	Vulnerability mitigations	Identify and mitigate known vulnerabilities in the selected technology stack (dependency pinning, secure defaults, randomness source)
5.1.1	Primary agent / server	Implement the server-side ecosystem elements specified by Stage 3's services
5.1.2.1	Entity provisioning	Implement the physical/operational provisioning needs of constrained entities (power, connectivity, etc.)
5.1.2.2	Identity storage	Implement the Stage 3.2.2 identity-storage design on the selected platform
5.1.2.3	Entity firmware/software modules	Implement the functional modules (identity, Root of Trust, communication, and domain-specific services) for each entity type
5.2.1	Secure development	Apply secure-coding practices (no persisted secrets beyond what Stage 3 permits, strong randomness, no hardcoded credentials)

Table S4: Phase D, Stages 6–7: Deployment, operation, and maintenance — generic specification.

Process ID	Parameter	Generic Requirement / Expected Output
6.1.3	RoT execution	Execute the Stage 2.2.3 Root of Trust process end-to-end on real entities; confirm mutual identification before any operational communication
6.2.1	Functional test coverage	Define and execute functional tests covering every implemented module; specified-but-unimplemented modules are marked not applicable, not passed/failed
6.2.2	Tamper verification	Verify that deliberate tampering with persisted identity parameters is detected as an authentication failure
7.1.1	Monitoring	Monitor entity authentication status, session history, and (where implemented) operational telemetry
7.1.4	Maintenance cycles	Track whether, and what, M1/M2/M3 maintenance points have fired
7.2.1	Anomaly detection	Specify an anomaly/trust-scoring mechanism that downgrades or isolates entities exhibiting abnormal behavior
7.2.2	Update elements	Specify the secure-update mechanism (authentication plus integrity requirement for updates)
7.2.3	Incident response	Specify the response procedure triggered by repeated authentication or anomaly-detection failures
7.2.4	Security-informed domain coordination	Specify how the entity-level anomaly/trust signal (Process 7.2.1) feeds back into the ecosystem's domain-specific coordination logic, where applicable

Table S5: Phase A, Stage 1: Organizational aspects — SMG application.

Process ID	Parameter	SMG Value
1.1	Problem statement	Manage distributed DC renewable energy generation, storage, and consumption across 2 co-located laboratory nodes; provide global coordination and real-time monitoring without centralized national grid
1.1.1	Cost-benefit justification	Energy losses (imbalanced generation/demand) eliminated; TRL 3 budget: 2× ESP32 NodeMCU + RPi4 + commercial solar adapters + AGM batteries + IBFIA SSRs
1.1.2	Entities involved	5: IoT nodes (2× ESP32), IoT server (RPi4), users (operators), administrators (researcher), DC bus (physical)
1.3.1	Development iteration (TRL)	ITER 1 target: TRL 3 (proof of concept with physical elements, controlled laboratory)
1.4.1	Criticality classification	Critical infrastructure — electrical energy; high-security tier selected (Stage 1.4.1)
1.4.2	Technological sovereignty	No cloud services; all computation on ESP32 (local) and RPi4 (global); open-source software only
1.4.3	Data sovereignty	Self-hosted RPi4 server; no third-party cloud; all sensor data stored locally
1.4.4	Access control policy	Token-based; nodes: POST sensor data only; users: GET monitoring; admins: full configuration and entity management
1.4.5	Communication policy	Mandatory application-layer encryption (AES-256-GCM); TCP over WiFi 802.11 b/g/n; no plaintext data channel
1.4.6	Applicable guidelines	Secure-by-Design principles (UK Government Security, 2024); NIST IoT security (Fagan et al., 2021); OWASP IoT Top 10

Table S6: Phase B, Stages 2–3: Architecture and service design — SMG application.

Process ID	Parameter	SMG Value
2.1.4	Communication topology	Centralized (hub-and-spoke): both nodes communicate exclusively with primary server; no node-to-node direct communication
2.1.5	Communication infrastructure	WiFi 802.11 b/g/n; ESP32 built-in; TCP sockets; RPi4 configured as WiFi AP (SSID: SMG_Primary; NetworkManager hotspot, 10.42.0.0/24 with gateway 10.42.0.1)
2.1.7	Processing topology	Fog topology specified: local 9-rule fuzzy EMS on each ESP32; global 27-rule security-aware fuzzy EMS on RPi4; neither EMS is implemented in the current firmware/server (see main text Limitations). Commercial solar adapter handles droop and battery-charging voltage regulation autonomously (implemented, independent of the EMS gap)
2.1.8	Entity hierarchy	IoT Nodes < Users < Administrators
2.2.2	Authentication mechanism	Nonce-bound asymmetric (ECDSA) PUF-seeded device proof + per-node symmetric (SHA-256 hash-reconstruction), conventionally-provisioned server proof for IoT nodes; plain SHA-256 hash for users/admins (not a salted, slow key-derivation function; see main text Limitations)
2.2.3	Root of Trust process	PUF-seeded scheme (`esp32_puflib`): enrollment via RTC-SRAM measurement (admin workstation) → non-secret mask/ECC helper data to NVS (USB serial) → HKDF-derived EC key pair → device public-key registration and per-node server-verifier provisioning → per-boot response reconstruction and mutual identification with an ephemeral ECDH key exchange
2.2.4	Secure communications	AES-256-GCM (ESP-IDF `mbedtls`/`esp_aes`) with 12-byte random nonces per message, keyed from $Key_{ab} = \text{SHA-256}(Z \parallel V_B \parallel N_A \parallel N_B)$ (Z a discarded per-session ECDH secret, Table 3 main text), not solely PUF-derived; no message-level cryptographic chain (design trade-off, see main text Limitations); no TLS certificate management needed on constrained devices
3.1.1	Services defined	4 specified: sensor data ingestion (nodes→server), EMS coordination (server→nodes), authentication API (all entities), monitoring dashboard (server→users); only the RoT-based node authentication service is implemented in the current firmware/server (see main text Limitations)
3.2.1	Access control	Entity type determined by identification channel (main text Section 5.6), not a session-token field; a separate token-based RBAC API for users/admins is specified but exists only in the original, not-yet-ported server module set (see main text Limitations)
3.2.2	Identity storage	ESP32: NVS flash partition (separate from firmware, survives updates, JTAG-disabled for TRL 5+); Server: PostgreSQL-equivalent for SMG + `puf_node_registry.json` for provisioning records

Table S7: Phase C, Stages 4–5: Implementation technologies and functional elements — SMG application.

Process ID	Parameter	SMG Value
4.1.1	TRL 3 criteria met	Physical node operational; mutual authentication functional; all applicable functional tests passed (Table 15, main text). PWM energy control and sensor-data logging are specified by the physical hardware (Table S7, process 5.1.2) but not yet exercised by the current C/ESP-IDF firmware, which implements only identity/RoT/secure-communication (see main text Limitations)
4.1.2.1	Node platform	ESP32 + C/ESP-IDF (v5.4.2, per the flashed firmware build log): built-in WiFi, SHA-256 HW acceleration, NVS; `esp32_puflib` (Stanicek, 2022) linked directly as a native ESP-IDF component (no language-runtime wrapper needed); identity/RoT/communication firmware (`smg_puf_rot_node`) builds cleanly under ESP-IDF; flashed, enrolled, and bench-validated end-to-end on SMG_NODE_01 ($N = 30$ real hard-reset trials, 30/30 success) — SMG_NODE_02 bench status still pending
4.1.2.2	Cryptography	HKDF-SHA256 (RFC 5869) derives a P-256 (secp256r1) key pair from the PUF response (`esp32_puflib`); device authenticates via ECDSA signature over a server-issued nonce (`mbedtls`), verified against the registered public key; server authenticates via a per-node SHA-256 hash-reconstruction proof; session-key derivation $Key_{ab} = \text{SHA-256}(Z \| V_B \| N_A \| N_B)$, Z a discarded per-session ECDH secret (`rot_session.c`); AES-256-GCM via ESP-IDF `mbedtls/esp_aes`
4.1.3	Communication protocol	TCP over WiFi 802.11 b/g/n; application-layer nonce-based mutual authentication protocol (implementation-specific Stage 4 choice, presented as part of this work's own case study)
4.2.1	Vulnerability mitigations	ESP-IDF v5.4.2 pinned (per the flashed firmware build log); WiFi SSID/password provisioned into NVS out-of-band (`tools/provision_server_ref.py`), no longer a firmware-source constant (see main text Limitations for the residual WPA2-PSK fleet-sharing property); all AES-GCM nonces and challenge values from `esp_fill_random()` (ESP-IDF HW RNG); no persistent device secret exists to protect, since the device's private key is derived fresh each boot and never stored; JTAG disable deferred to TRL 5+
5.1.1	Primary agent (RPi4)	Currently active: `puf_server.py` + `rot_challenge_service.py` (PUF-based mutual authentication challenge service). `server.py`, `auth_service.py`, `ems_coordinator.py` (27-rule security-aware fuzzy), `data_store.py` are the original module set, specified but not yet ported to the current RoT server (main text Limitations)
5.1.2.1	Node power supply	DC-DC buck converter (12V→5V) per node from local battery; isolates ESP32 from shared DC bus; ≈90% efficiency vs. ≈27% for LDO at 12 V (avoids ≈3.8 W peak dissipation)

Table S8: Phase D, Stages 6–7: Deployment, operation, and maintenance — SMG application.

Process ID	Parameter	SMG Value
6.1.3	RoT execution	PUF-based scheme, executed and validated on SMG_NODE_01 (1/2 nodes; SMG_NODE_02 pending): admin runs `enroll_puf()` → mask/ECC helper data written to ESP32 NVS → server verifier registration via `register_puf_node.py` → per-boot reconstruction and nonce-bound mutual identification confirmed over $N = 30$ real hard-reset trials (30/30 success)
6.2.1	Functional test coverage	6 tests (Table 15, main text): boot/PUF reconstruction (FT-01, 30/30), WiFi connect (FT-02, 30/30), P2 handshake (FT-03, 30/30), P3 session (FT-04, 30/30), tamper verification (FT-05, 1/1), cross-chip cloning negative test (FT-06, 1/1); fuzzy-EMS and sensor-specific tests are not applicable, since neither module is implemented in the current firmware
6.2.2	Tamper verification	NVS parameter tampering confirmed as authentication failure (single targeted negative test; full penetration testing deferred to ITER 2)
7.1.1	Monitoring (Stage 7.1.1)	Authentication status and session history (evidentiary log, no cryptographic chain, see main text Limitations), from `rot_challenge_service.py`'s server-side logs. Per-node real-time $V_{bus}/I_{bus}/P_{bus}/V_{gen}/I_{gen}/P_{gen}/D_{pwm}$ telemetry and heap-free monitoring are specified but not yet implemented in this firmware (no sensor module, see main text Limitations)
7.1.4	Maintenance cycles	One maintenance moment (SMG_NODE_01): an enrollment-state-confusion bug causing an unbounded re-enrollment loop on reconstruction retry, identified and fixed via code-level revision (M2) prior to the $N = 30$ bench trial. M1 and M3 not triggered at TRL 3; SMG_NODE_02 maintenance-cycle evidence remains pending.
7.2.1	Anomaly detection	Per-node trust score (float 0.0–1.0) specified: initialized 0.80 on successful mutual authentication; −0.30 on authentication-tag failure; −0.40 on V_{bus} anomaly; UNTRUSTED (< 0.30) ⇒ TCP dropped; recovery requires re-auth. Implemented in the original server module set's own phase numbering (`auth_service.py`, its P3 session-establishment step, distinct from the current protocol's P2/P3); not yet ported to `rot_challenge_service.py` (see main text Limitations)
7.2.2	Update elements	OTA update requires admin token + firmware signature (deferred to ITER 2)
7.2.3	Incident response	Auth failures logged; > 3 consecutive failures ⇒ quarantine specified; same not-yet-ported status as 7.2.1 (see main text Limitations)
7.2.4	Security-aware EMS	Global fuzzy EMS (27 rules) on RPi4 specified to accept the per-node trust score from 7.2.1 as a third input alongside voltage tracking error and bus current; neither this EMS nor 7.2.1's trust scoring is implemented in the current RoT server (see main text Limitations)